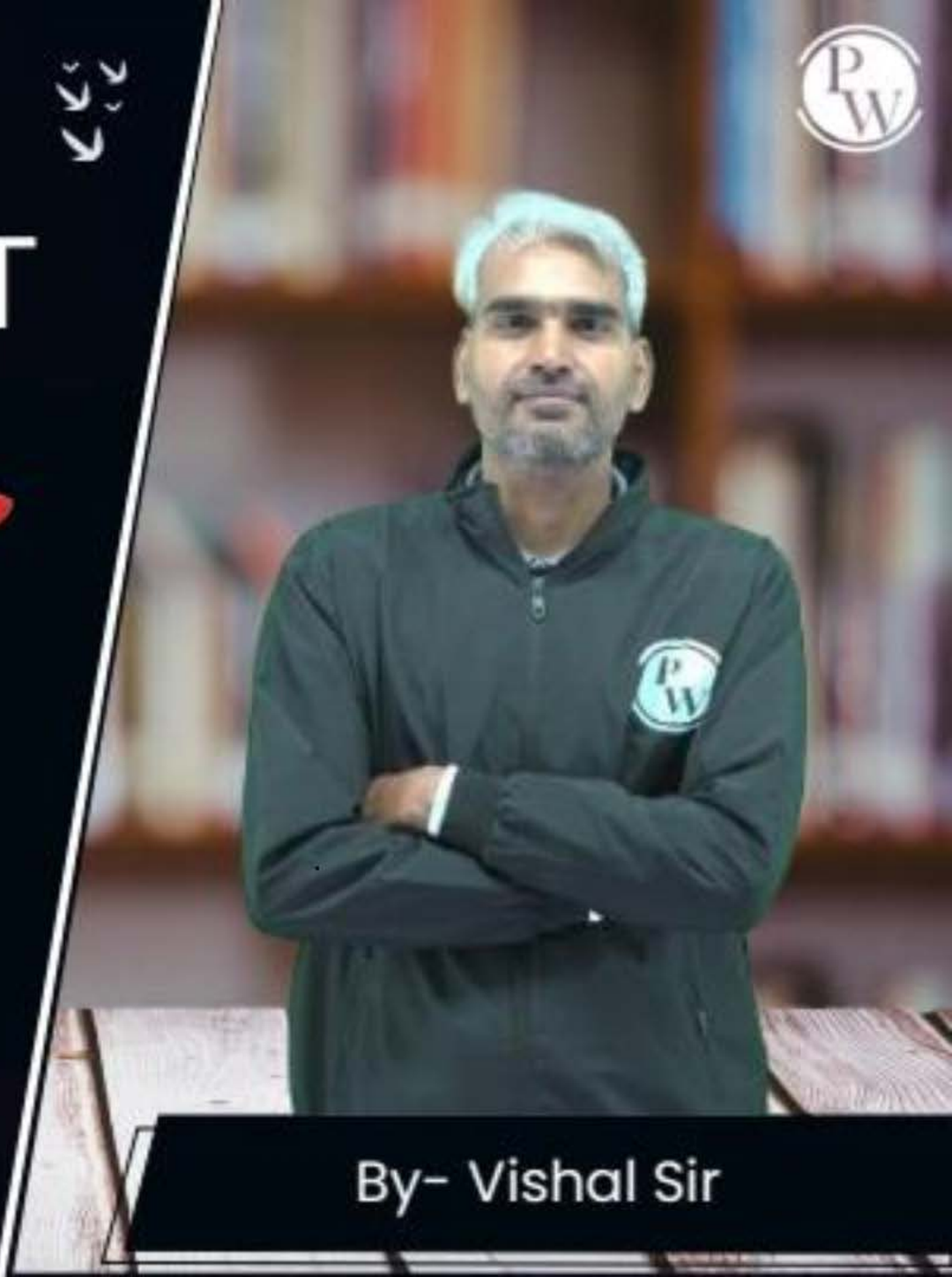
Computer Science & IT



Database Management System

Transactions & concurrency control

Lecture No. 12



By- Vishal Sir

Recap of Previous Lecture



- Topic: Basic 2PL and lock upgrading/downgrading
- Topic: Problems possible with Basic 2PL
- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL

Slide

Topics to be Covered



- Topic: Strict 2PL
- Topic: Conservative 2PL
- Topic: Rigorous 2PL
- Topic: Time stamp
- Topic: Read time stamp & Write time stamp
- Topic: Basic time stamp ordering protocol

Slide



Topic : Basic 2PL with lock upgrading

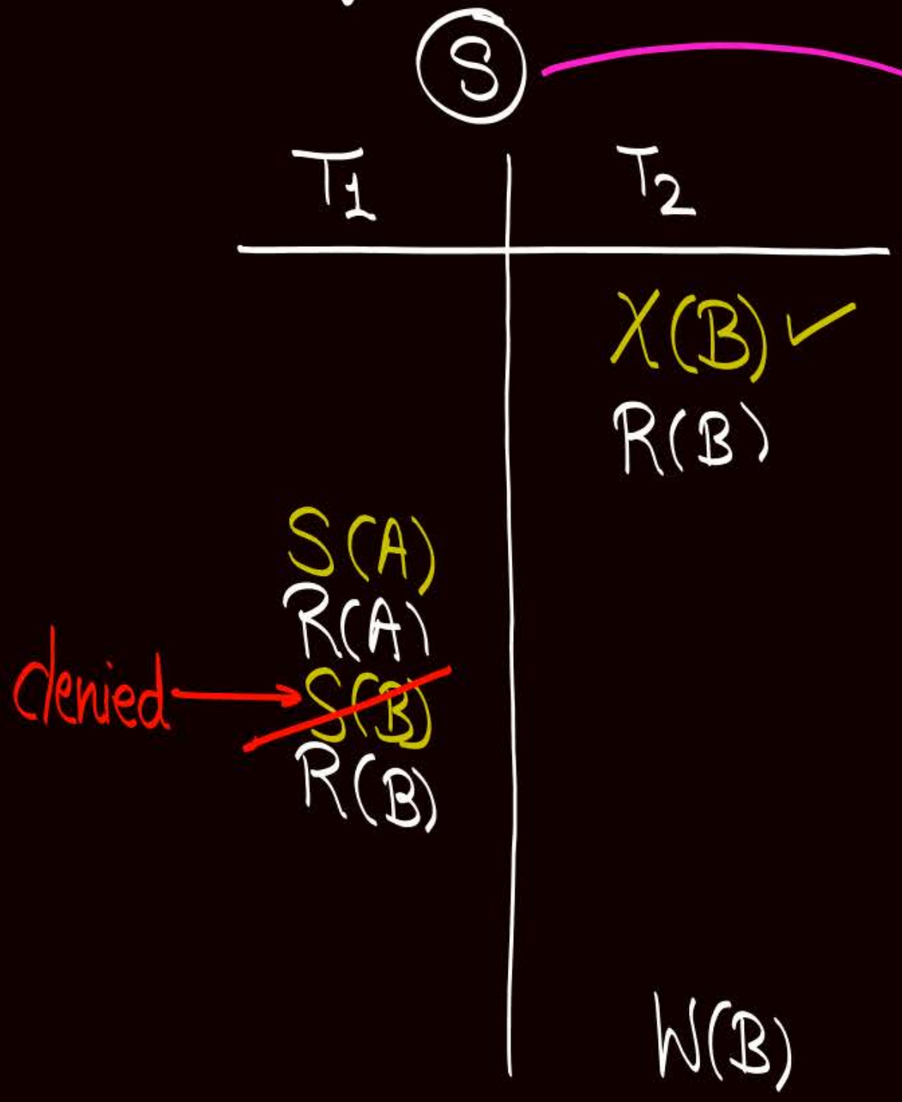
If lock conversion (upgrading/downgrading) is allowed, then we can upgrade shared lock into exclusive locks in the Growing Phase, and we can downgrade from the exclusive lock to shared lock in the shrinking phase.

Upgrade ⇒ Shared to Exclusive

Downgrade ⇒ Exclusive to Shared

* We are not allowed to upgrade if we are already in the shrinking phase
* If transaction is not already in its shrinking phase, then also we can downgrade the lock, but as soon as we downgrade shrinking phase will start.

Q:- Check whether the following schedule is allowed to execute using Basic 2PL or not ?



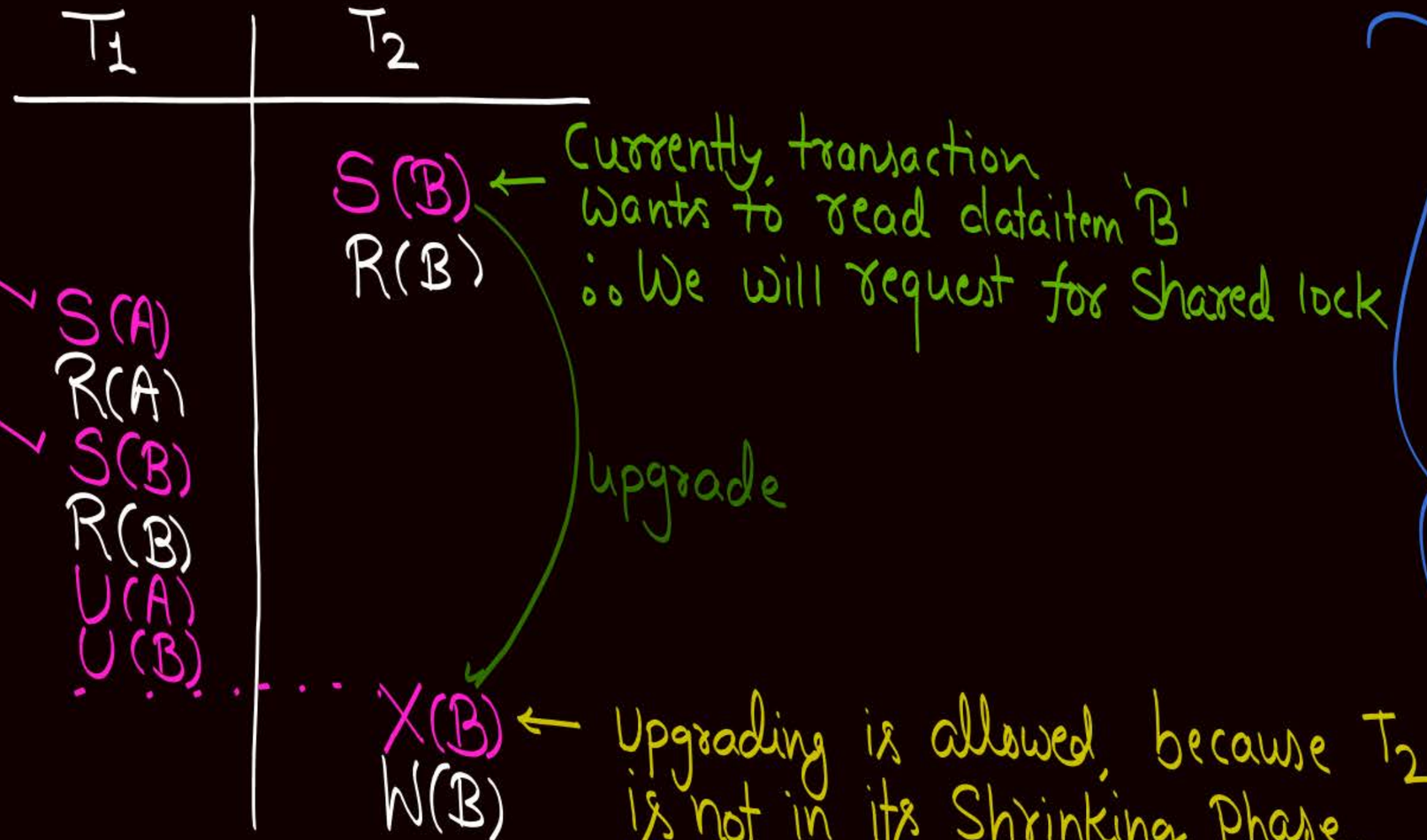
It is a Conflict Serializable Schedule

↓

and it is not allowed to execute using Basic 2PL (without lock upgrade)

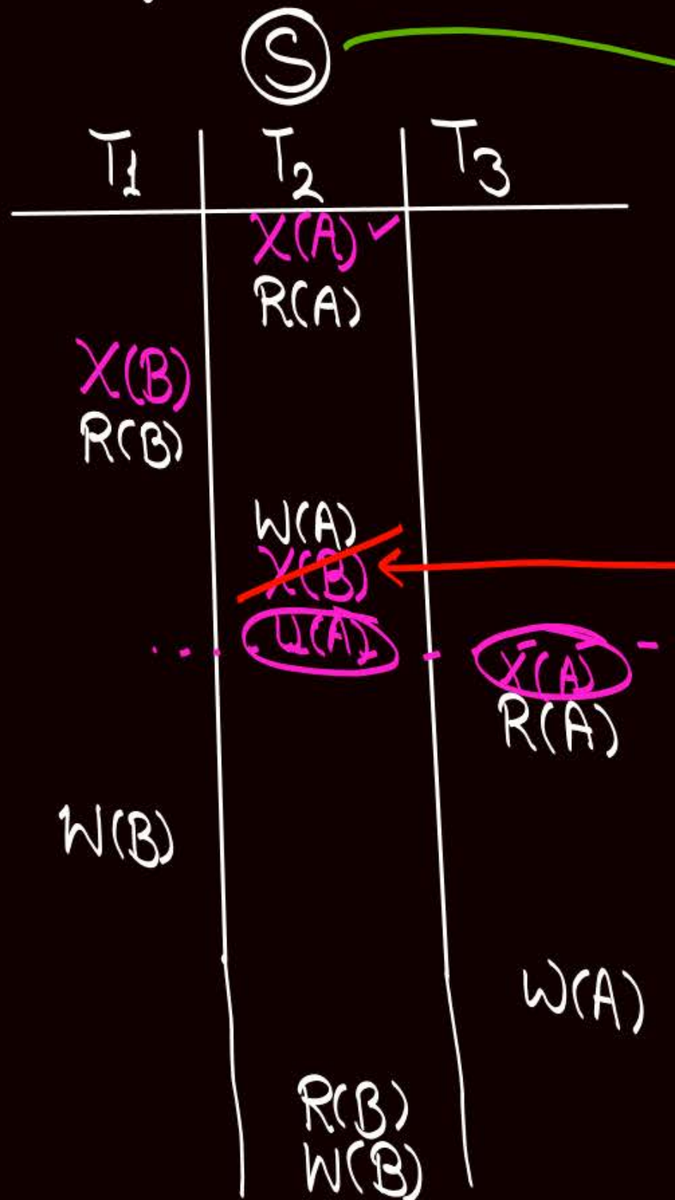
Q:- Check whether the following schedule is allowed to execute using Basic 2PL with lock upgrading or not?

(S)



It is a C.S.S. which is not allowed by Basic 2PL without lock upgrade but it is allowed by Basic 2PL with lock upgrade

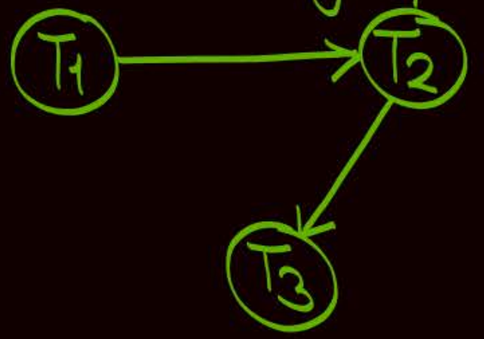
Q:- Check whether the following schedule is allowed to execute using Basic 2PL or not ?



denied

∴ Not allowed by Basic 2PL without lock upgrade

Precedence graph

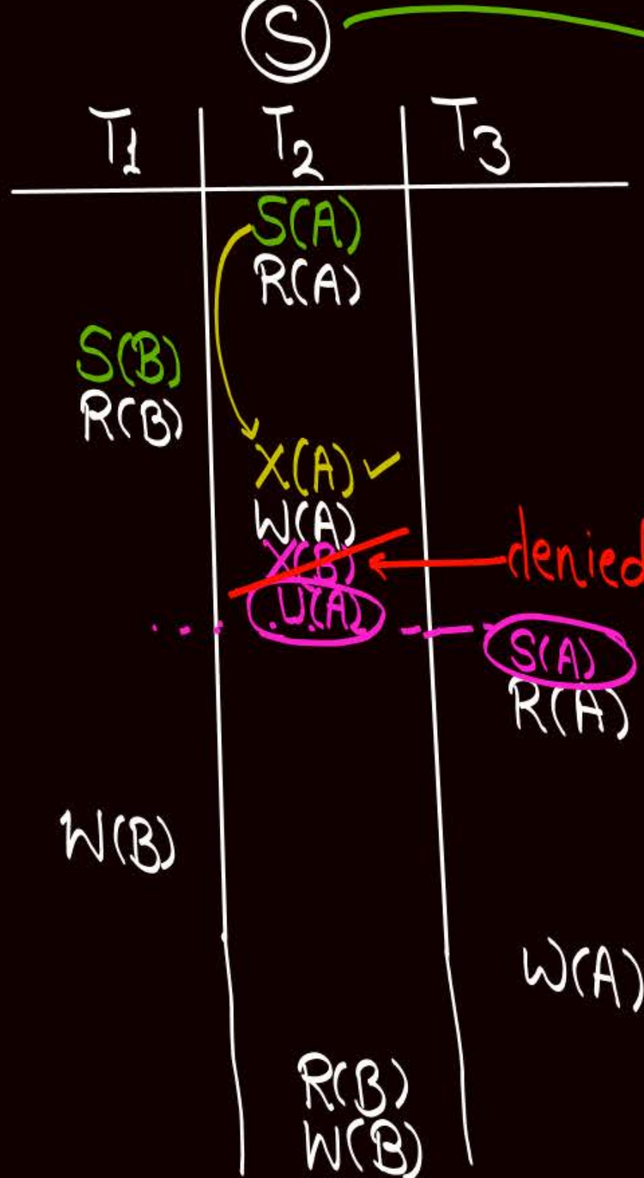


⇒ Acyclic

∴ C.S.S.

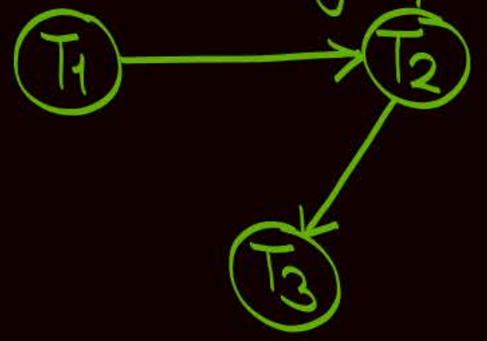
but not allowed by Basic 2PL without lock upgrading

Q:- Check whether the following schedule is allowed to execute using Basic 2PL with lock upgrading or not?



∴ Not allowed by Basic 2PL even when lock upgrading is allowed

Precedence graph



⇒ Acyclic

∴ C.S.S.

but not allowed by Basic 2PL even when lock upgrading is allowed

Note:- ① If lock upgrading is allowed, then we may execute some extra conflict serializable schedules Compared to Basic 2PL without lock upgrading, but still there remains some conflict serializable schedules that are not allowed by Basic 2PL even when lock upgrading is allowed

② If schedule is not a conflict serializable schedule then it is never allowed by any type of 2PL, does not matter whether lock upgrading is allowed or not.



Topic : Problems possible with Basic 2PL

A schedule that is allowed to execute using basic 2PL protocol may suffer from,

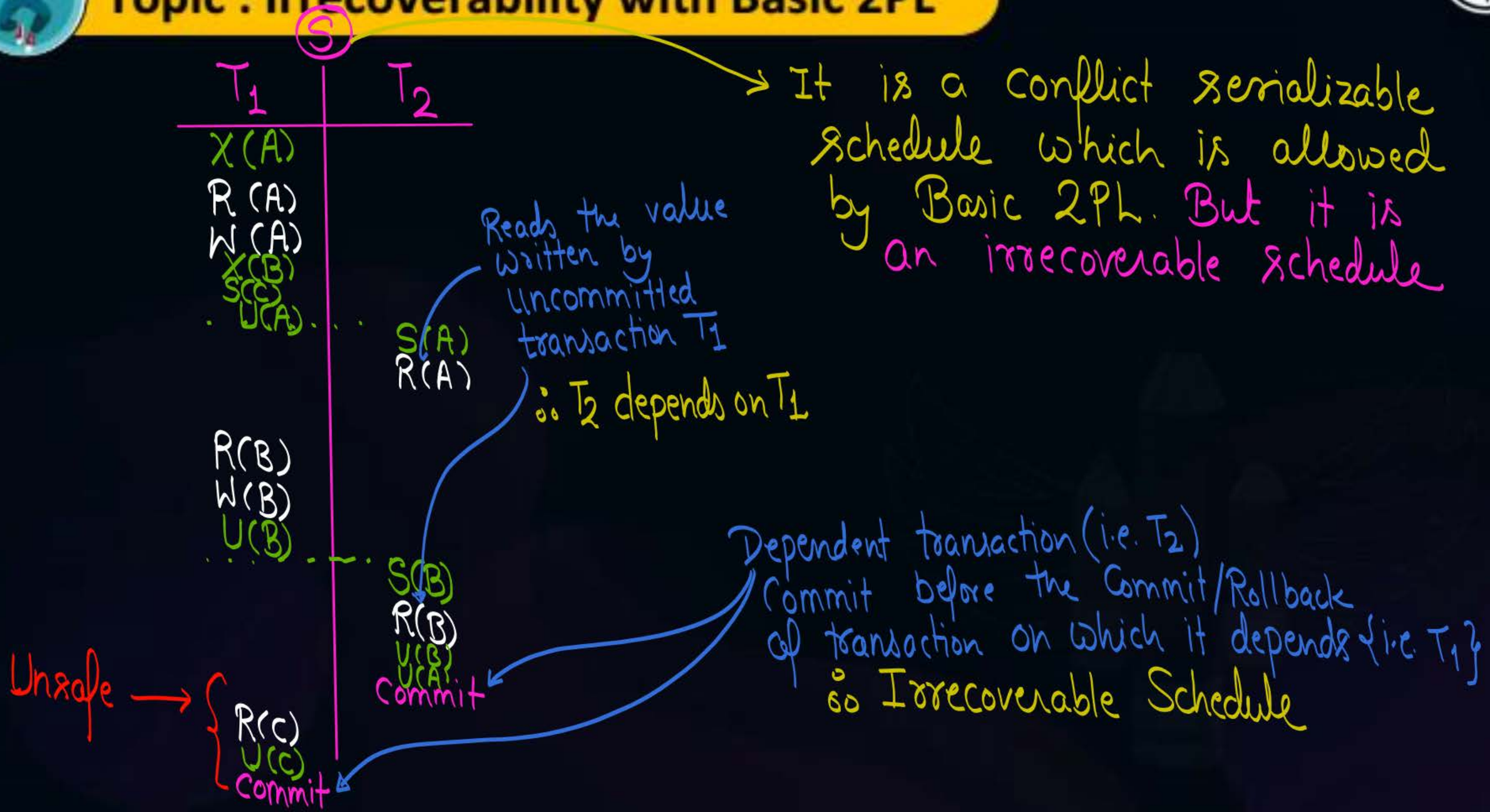
✓ ① Irrecoverability {Can be solved using Strict-2PL}

✓ ② Deadlock {Can be solved using Conservative 2PL}

✓ ③ Starvation {No solution}

Topic : Irrecoverability with Basic 2PL





It is a conflict serializable schedule which is allowed by Basic 2PL. But it is an irrecoverable schedule

Reads the value written by Uncommitted transaction T_1

$\therefore$ T_2 depends on T_1

Dependent transaction (i.e. T_2) Commit before the Commit/Rollback of transaction on which it depends {i.e. T_1}

$\therefore$ Irrecoverable Schedule

Topic : Deadlock with Basic 2PL



S

T_1	T_2
S(A)	
R(A)	
	X(B)
	W(B)
S(B)	
R(B)	
	X(A)
	W(A)

T_1 requires the resource held by T_2

denied → S(B)

Time out

T_2 requires the resource held by T_1

X(A) ← denied

Time out

Dependency graph



it is cyclic

∴ Deadlock possible



Topic : Starvation with Basic 2PL

T_1	T_2	T_3	T_4	T_5
	S(A)			
denied because of T_2 → X(A)				
Time out	U(A)	S(A)		
denied because of T_3 → X(A)				
Time out		U(A)		
denied because of T_4 → X(A)			X(A)	
Time out			U(A)	
denied because of T_5 → X(A)				S(A)

If this keeps happening, then we say that transaction T_1 is under Starvation



Topic : 2PL Classification

1. Basic 2PL {Already done}
2. Strict 2PL
3. Conservative 2PL
4. Rigorous 2PL



Topic : Strict 2PL

{ 2PL restriction + Strict Recoverability Condition } ≡ Strict-2PL

2PL restriction:

A transaction T can request for a lock only if it has not performed any unlock oph

+

Strict Recoverability Condition:

T_1	T_2
W(A)	
⋮	⋮
Commit/Rollback	
	R(A)/W(A)

≡

Strict-2PL:

T_1	T_2
X(A)	
⋮	
Commit U(A)	
	S(A)/X(A)

it will ensure Serializability

Free from ① Irrecoverability
② Cascading Rollback
③ Lost-update



Topic : Strict 2PL

Strict 2PL, is 2PL protocol with restriction that Every Exlusive lock acquired by the transaction must be unlocked only after the Commit opn of that transaction.

Shared locks can be unlocked at any point

Strict-2PL

T_1	T_2
X(A)	
⋮	
Commit U(A)	
	S(A)/X(A)



Topic : Strict 2PL

Strict 2PL is,

- Free from,
  1. Irrecoverability
  2. Cascading Rollback
  3. Lost-update
- Not free from,
  1. Deadlock
  2. Starvation

Strict-2PL

T_1	T_2
X(A)	
⋮	
Commit U(A)	
	S(A)/X(A)



Topic : Conservative 2PL

{ Used to solve the Problem of deadlock }

* In conservative 2PL, we will dis-satisfy "Hold & wait" by "Hold or Wait"
* In 'Conservative - 2PL' transaction will request for all the locks required for its execution before starting its execution.

i.e. transaction will hold all the locks & wait for None

* If all the locks requested by the transaction are granted then only transaction will start its execution, in this case transaction will not have to depend on any other transaction for locks during its execution

i.e, tanasaction will wait for the locks and holds none

* If any one of the requested lock is not granted, then it will release all the granted locks as well {i.e. it will not hold any lock}, and it will go into time out, once time out period is over it will again request for all locks

No other transaction will depend on it

{ Necessary Conditions for deadlock.
1. Mutual Exclusion
2. No preemption
3. Hold & Wait
4. Circular Wait

⇓

If any of the necessary Condⁿ Can be dis-satisfied then there will be no deadlock



Topic : Conservative 2PL

{ Used to solve the Problem of deadlock }

* In conservative 2PL, we will dis-satisfy "Hold & wait" by "Hold or Wait"

* Conservative 2PL will only define the order in which locks can be acquired {i.e, before starting the execution of transaction}, but it does not define the order in which locks must be released.

{i.e, in Conservative 2PL, an Exclusive lock acquired by a transaction may be unlocked, before the Commit oph of that transaction}

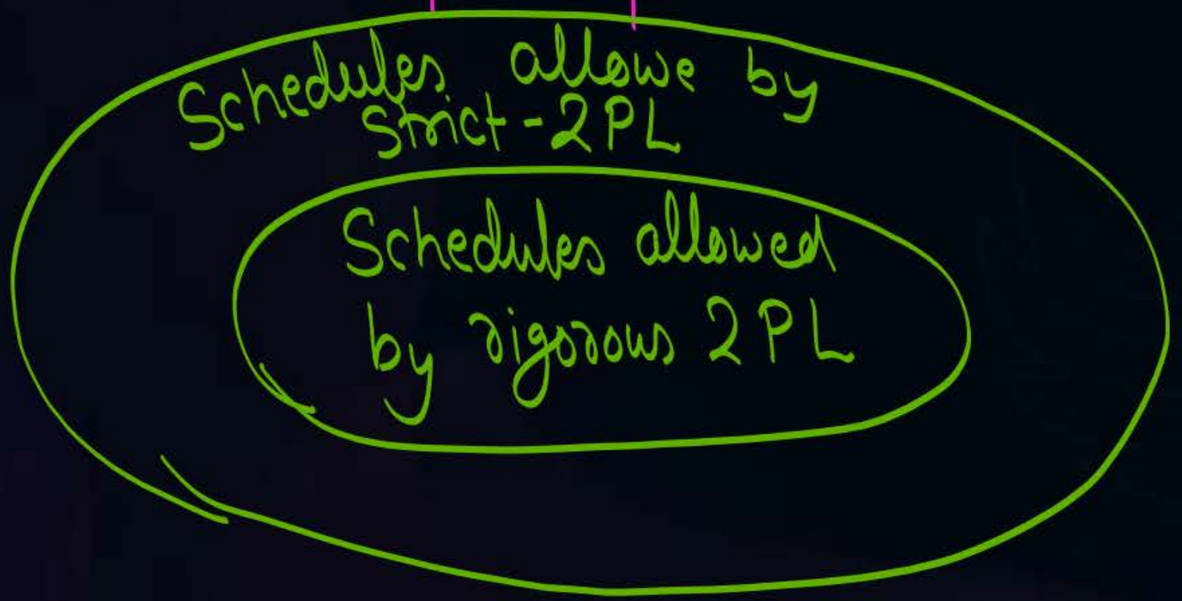
Hence, Conservative 2PL may suffer from irrecoverability, Cascading rollback and lost-update Problem.

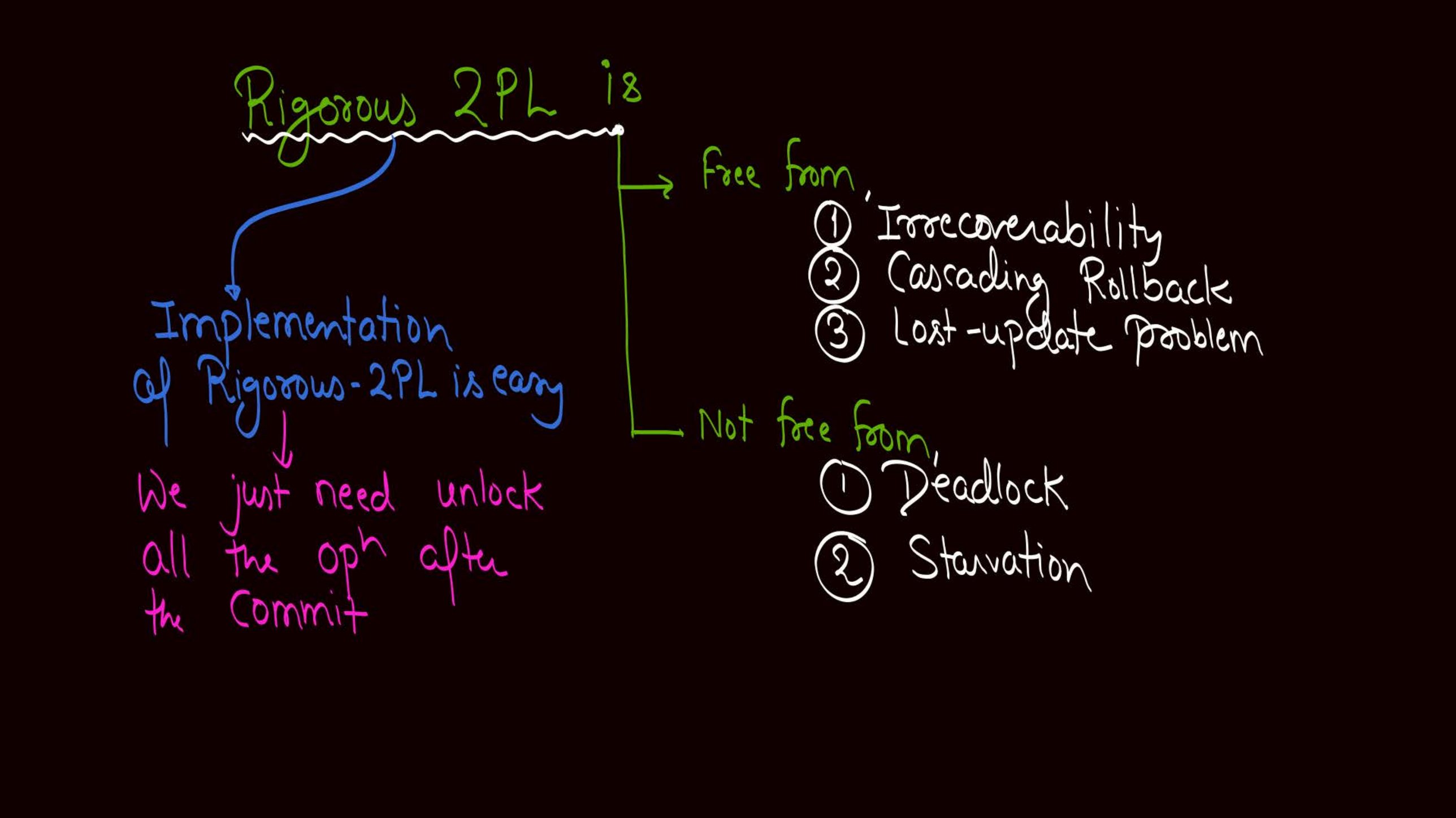
* Conservative-2PL is free from deadlock, but starvation is still possible {actually Starvation may increase}



Topic : Rigorous 2PL

* In Rigorous 2PL, both Shared (S) as well as Exclusive (X) locks must be unlocked after the Commit op^{n} of transaction




Rigorous 2PL is
Implementation of Rigorous-2PL is easy
We just need unlock all the oph after the Commit
Free from,
1 Irrecoverability
2 Cascading Rollback
3 Lost-update problem
Not free from,
1 Deadlock
2 Starvation

Time Stamp Ordering Protocols



Topic : Time stamp ordering protocols

- There are two different versions of time stamp ordering protocol
  1. Basic time stamp ordering protocol
  2. Time stamp ordering protocol with Thomas Write Rule

* Time Stamp :-

Time Stamp is a unique value assigned by DBMS to each transaction in ascending order.

* Let T_1 and T_2 are two transactions in the system, such that

Time Stamp of T_1 $<$ Time Stamp of T_2
$TS(T_1)$ $TS(T_2)$

then, T_1 is the old transaction
&
T_2 is the younger transaction

- Read time stamp of data item 'A'

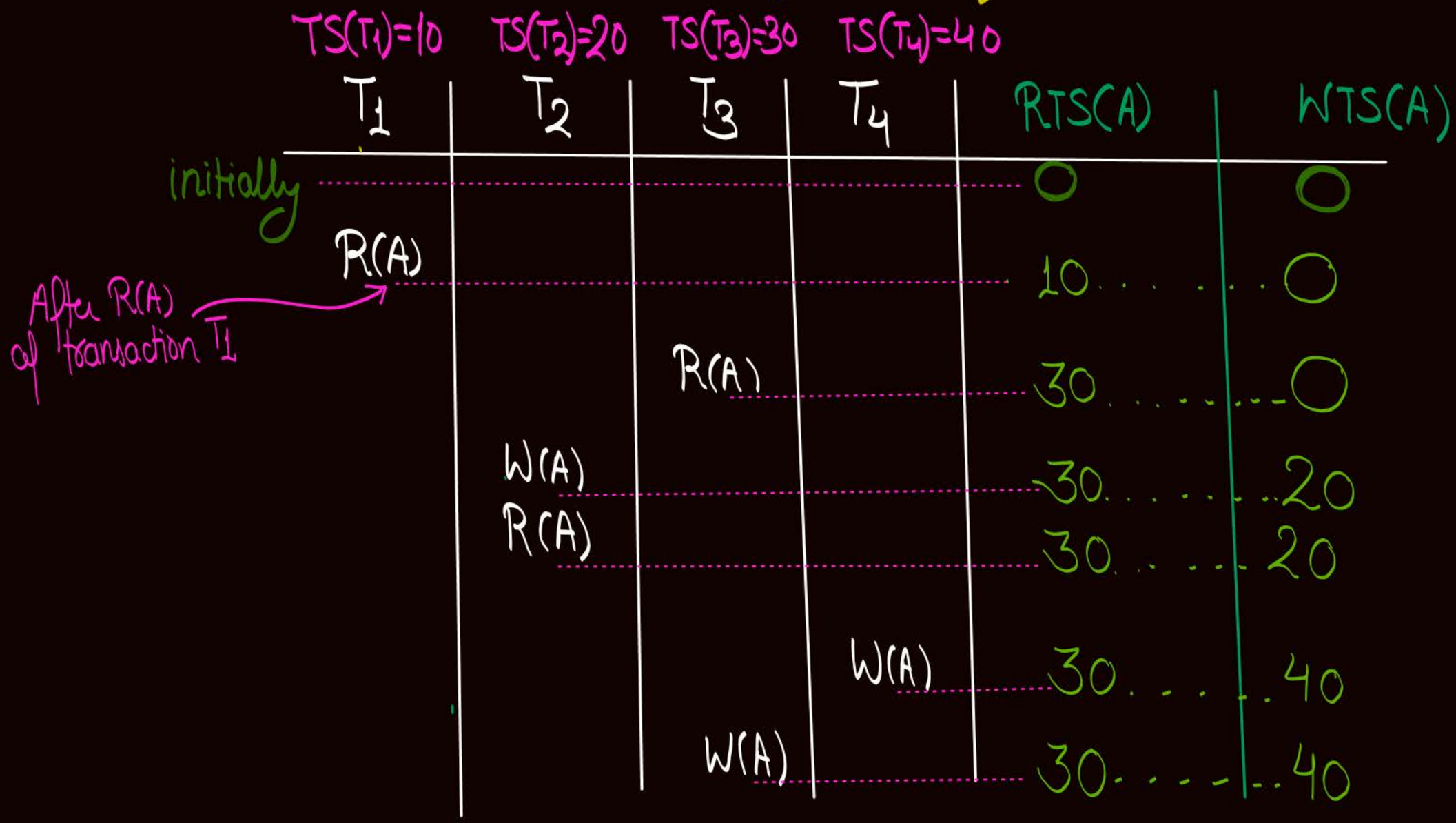
  RTS(A): It is the highest time stamp value among the time stamps of the transactions that has Performed Read(A) opn successfully.

- Write time stamp of dataitem 'A'

  WTS(A): It is the highest time stamp value among the time stamps of the transactions that has Performed Write(A) opn successfully.

* Initially, RTS(A) = 0, WTS(A) = 0 } for all dataitems, not only for 'A'.

eg: RTS(A) & WTS(A) :-


TS(T1)=10
TS(T2)=20
TS(T3)=30
TS(T4)=40
T1
T2
T3
T4
RTS(A)
WTS(A)
initially
0
0
R(A)
10
0
After R(A) of transaction T1
R(A)
30
0
W(A)
30
20
R(A)
30
20
W(A)
30
40
W(A)
30
40



2 mins Summary

- **Topic** Strict 2PL
- **Topic** Conservative 2PL
- **Topic** Rigorous 2PL
- **Topic** Time stamp
- **Topic** Read time stamp & Write time stamp
- **Topic** Basic time stamp ordering protocol

THANK - YOU